Data-Structures & Algorithms

with

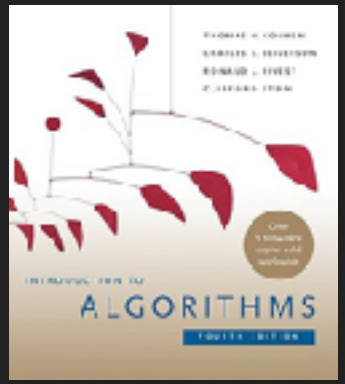
Manoj Prabhakaran

Lecture 3

Linked Lists

Course Logistics

- Bodhi Tree
  - We'll use this as a portal for all course-related activities
  - Discussion forum, announcements, labs, lecture slides
- Reference textbook: CLRS
  - We don't need everything in the book
  - And we'll cover a few things not in the book



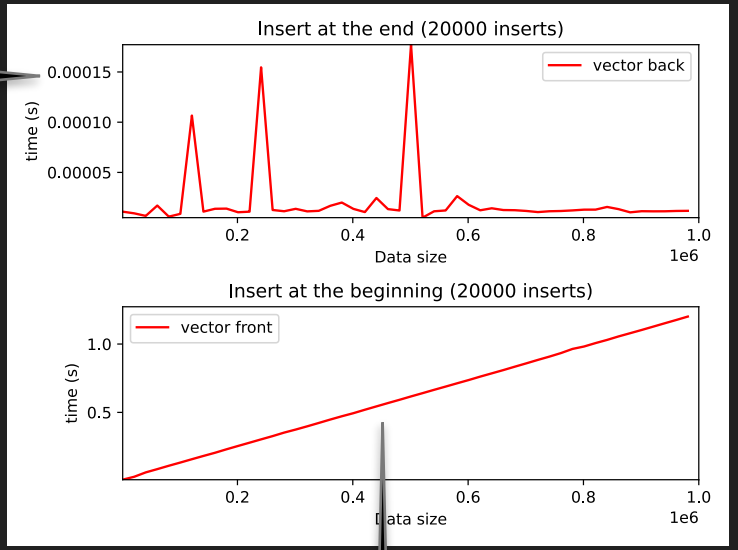
Arrays

- We saw std::vector has Θ(1) insertion (and Θ(n) search) time
- But that is only if you insert at the back of the vector
  - In std::vector, `v.push_back(x)`
- What happens if you insert elsewhere?
- A vector uses a contiguous array of memory locations to store its elements
- To insert at location i, you need to physically move the memory contents at locations i, i+1, i+2, … to make room for the new element
  - O(n-i) time, if n is the size of the vector
  - Can we really see this?

Inserting into an Array

```
std::vector<int> v1, v2;
...
// in a loop
  ...
  v1.insert(v1.begin(), 1); // add as v[0]
  v2.insert(v2.end(), 1); // v2.push_back(1)
...
```

microseconds vs 1 second!



Θ(1) vs Θ(n)

An Alternative to Arrays

- Linked lists: Maintain an ordering of the items, while still allowing O(1) insertion to the front and the back
- A list of “nodes” which do not have to be in contiguous memory locations in order
- Each node has a pointer to the next one in the list
  - The last node has `nullptr` as next

O(1) insertion

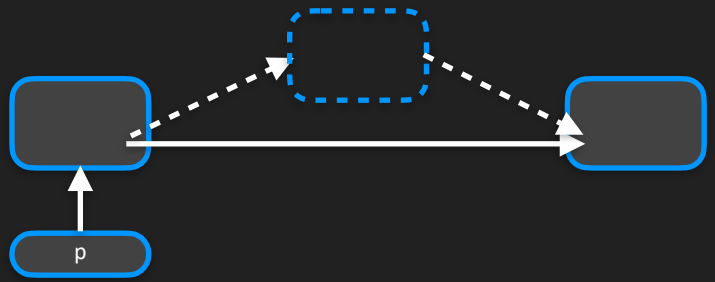
```
struct node { int val; node* next = nullptr; }; // a node in the list
int main() {
  node *head, *tail;
  head = tail = new node{0}; //a list with a single node
  // insert at the end
  tail->next = new node{1}; //attach a new node at the back
  tail = tail->next;        //point tail to the new node
  // insert at the beginning
  head = new node{-1,head};
}
```

```
node* tmp = new node;
tmp->val = -1;
tmp->next = head;
head = tmp;
```

O(1) insertion

- Can insert not just to the front and back, but anywhere in a list
  - Once we have a position in the list to insert into, insertion itself takes a fixed number of operations, irrespective of the size of the list

```
// p points to a node; to insert a new node with val x after that node
p->next = new node{x,p->next};
if(p==end) end = p->next;
```

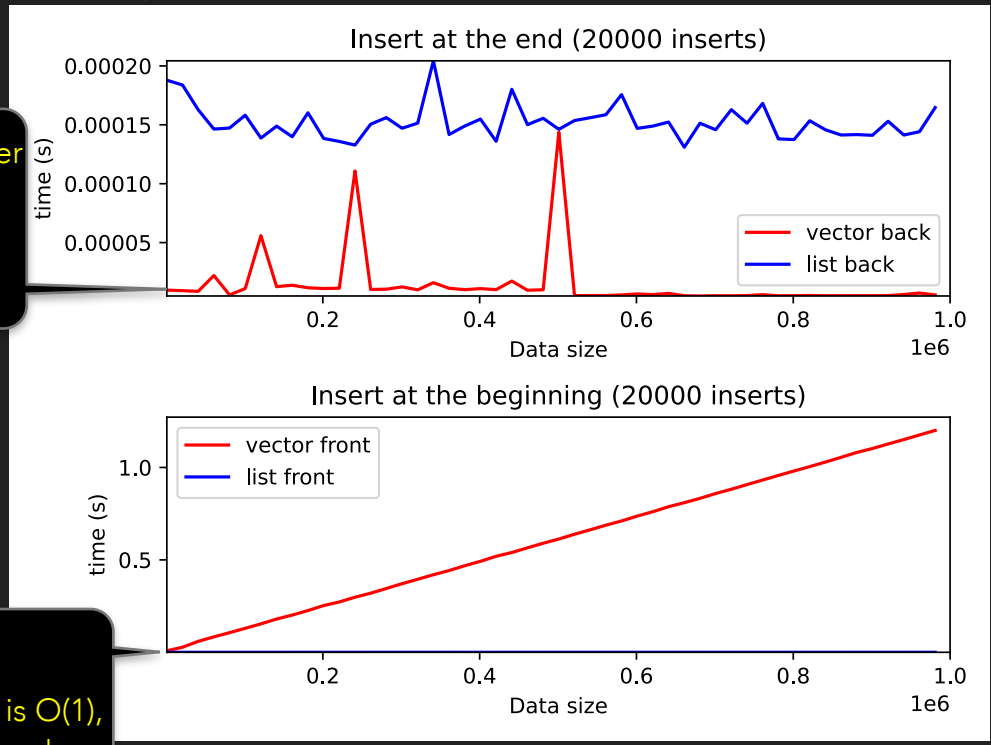



```
node* tmp = new node;
tmp->val = x;
tmp->next = p->next;
p->next = tmp;
```

List vs. Array Insertion

```
std::vector<int> v1, v2;
std::list<int> L1, L2;
...
// in a loop
  ...
  v1.insert(v1.begin(), 1);
  v2.insert(v2.end(), 1);
  L1.push_front(1);
  L2.push_back(1);
...
```

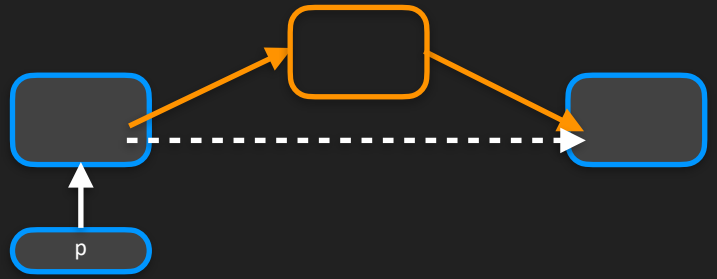
Vector is an order of magnitude faster for push_back



List insertion is O(1), at either end

O(1) Deletions

- In a list, deletions are also O(1)
- Need the link to the node before the one being deleted




```
// To remove p->next
p->next = p->next->next; // OK?
```

- Problem: Memory leak!

```
// The proper way to remove p->next
node* tmp = p->next; p->next = p->next->next; delete tmp;
if (tail==tmp) tail = p;  // To be strictly safe, do this before delete
```